

REVISIÓN INTERNA · DEFENSA TFM

Lo que las diapositivas dicen y lo que el código hace

Seis discrepancias graves, cada una con la captura exacta del repositorio que la demuestra.

Resumen: seis discrepancias graves

Afirmaciones de las diapositivas que el código actual contradice. Evidencias en las páginas siguientes.

1

Diap. 8 y 11

Agente 2 sin LLM

Se instancia sin LLM: el scoring es intents + embeddings + palabras clave.

2

Diap. 9 y 10

top_k = 5.000, no 3.500

El valor por defecto real es 5.000 y así se ejecutaron las pruebas.

3

Diap. 10

Ejemplo de la diap. 10

«Segunda división francesa» se filtra como LaLiga2 (Segunda española).

4

Diap. 9 y 13

Filtro de liga ignorado

Ligas con paréntesis (Escocia, Suiza, Turquía...) se descartan sin avisar.

5

Diap. 12 y 13

El juez no ve los datos

GPT-4o no recibe las estadísticas y sí el nombre del modelo evaluado.

6

Diap. 5 y 7

Reproducibilidad y cifras

scouting-etl falla al arrancar; el último ETL da 15.066 jugadores, no 15.347.

1

El Agente 2 no usa el LLM

Afecta a: diapositiva 8 (diagrama y preguntas) y diapositiva 11 (scoring explicable)

LA DIAPOSITIVA DICE

«Agente 2 · LLM interpreta, score cuantitativo» · «El LLM extrae los conceptos futbolísticos y las familias de métricas relevantes.»

EL CÓDIGO HACE

- El pipeline crea ScoreEvaluatorAgent() sin llm; el Agente 1 sí lo recibe (línea 183, en verde).
- Con self.llm = None la extracción devuelve listas vacías y no llama a ningún modelo.
- Los conceptos salen de un diccionario de palabras clave; en porteros la rama devuelve {} siempre.

QUÉ CAMBIAR

Describir el Agente 2 como intents + embeddings + diccionario de conceptos. El LLM solo está en Supervisor, Agente 1 y Agente 3.

```
src/scouting/agents/pipeline.py líneas 182-190  
  
182 def nodo_agente1(state: PipelineState) -> PipelineState:  
183     a1 = Agente1HardFilter(llm=llm)  
184     df_filt, tipo = a1.filtrar(state["df_pre_filtrado"], state["query_norm"], state["tipo"])  
185     return {**state, "df_filtrado": df_filt, "tipo": tipo}  
186  
187 def nodo_agente2(state: PipelineState) -> PipelineState:  
188     a2 = ScoreEvaluatorAgent()  
189     resultados, df_top3 = a2.score_dataframe(state["df_filtrado"], state["query_norm"], state["tipo"])  
190     return {**state, "resultados": resultados, "df_top3": df_top3}
```

```
src/scouting/agents/agent2.py líneas 579-581, 651-662  
  
579     def extract_football_concepts_llm(self, query: str) -> Dict[str, Any]:  
580         if not self.llm:  
581             return {"concepts": [], "metric_families": []}  
:  
651     def _weights_via_llm(self, query: str, tipo: Literal["jugador", "portero"]) -> Dict[str, float]:  
652         if tipo != "jugador":  
653             return {}  
654  
655         extracted = self.extract_football_concepts_llm(query)  
656         concepts = extracted.get("concepts", [])  
657         families = extracted.get("metric_families", [])  
658  
659         if not concepts:  
660             concepts = self._find_concepts_fallback(query)  
661  
662         return self._concepts_to_metrics(concepts, families)
```

2 El top_k de recuperación es 5.000, no 3.500

Afecta a: diapositiva 9 (dos veces) y diapositiva 10 (diagrama y pregunta sobre FAISS)

LA DIAPOSITIVA DICE

«Top-k configurable: 3.500 perfiles por defecto.» · «En la implementación se fijó en 3.500 por defecto.»

EL CÓDIGO HACE

- El constructor del Agente 0 tiene top_k = 5000 y el pipeline no lo cambia.
- FAISS busca con self.top_k, es decir, 5.000 candidatos.
- Los logs reales lo confirman: todas las consultas empiezan con rows_iniciales=5000.

QUÉ CAMBIAR

Cambiar 3.500 por 5.000 en las tres menciones. Dato útil: en porteros (~1.500) se recupera la base entera.

```
src/scouting/agents/agent0.py líneas 70-72, 137-139  
  
70 class Agente0VectorRetriever:  
71     def __init__(self, top_k: int = 5000):  
72         self.top_k = top_k  
:  
137         emb = self.encoder.encode(query, normalize_embeddings=True)  
138         emb = np.array([emb]).astype("float32")  
139         D, I = index.search(emb, self.top_k)
```

```
logs_completos_tfm.txt líneas 16, 53  
  
16 [A1] query='Delantero de la liga escocesa con buen juego aéreo.' rows_iniciales=5000  
:  
53 [A1] query='Lateral derecho de la Eredivisie rápido y con buen centro.' rows_iniciales=5000
```

3 El ejemplo de la diap. 10 filtra otra liga

Afecta a: diapositiva 10 (ejemplo guiado); es la consulta 9 del Anexo D

LA DIAPOSITIVA DICE

«Interior ofensivo de segunda división francesa...» → Agente 1: restricción verificable «Segunda división francesa».

EL CÓDIGO HACE

- En la ejecución registrada, el LLM del Agente 1 devuelve liga = 'LaLiga2' (la Segunda española).
- Pasa en los tres bloques del log (líneas 335, 747 y 1162).
- El filtro deja solo 4 jugadores, todos de una liga que no es la pedida.

QUÉ CAMBIAR

Revisar qué devolvió esta consulta en Promptfoo. Si se repite, cambiar el ejemplo o presentarlo como caso de error.

```
logs_completos_tfm.txt líneas 318, 321, 335-336
318 [SUPERVISOR] domain=ok lang=es query_es=Interior ofensivo de segunda división francesa con buen último
    pase, alta producción de xA y contrato < 2 años...
...
321 [A1][LLM] criterios extraídos: {'edad_max': None, 'valor_max_euros': None, 'posiciones': ['attacking
    midfield'], 'edad_min': None, 'nacionalidad': None, 'pie_dominante': None, 'altura_min_cm': None,
    'altura_max_cm': None, 'contrato_max_anios': 2.0, 'liga': 'LaLiga2'}
...
335 [A1][Liga] detectadas=['LaLiga2']
336 [A1][Liga] filtro: 226 -> 4
```

```
logs_completos_tfm.txt líneas 7, 335, 412, 747, 826, 1162
7 ### EVALUANDO PROVEEDOR: OPENAI | MODELO: gpt-4o-mini
...
335 [A1][Liga] detectadas=['LaLiga2']
...
412 ### EVALUANDO PROVEEDOR: GROQ | MODELO: openai/gpt-oss-120b
...
747 [A1][Liga] detectadas=['LaLiga2']
...
826 ### EVALUANDO PROVEEDOR: OPENAI | MODELO: gpt-5.5
...
1162 [A1][Liga] detectadas=['LaLiga2']
```

4 El filtro de liga se ignora en silencio

Afecta a: diapositiva 9 («Filtros soportados: ... liga») y a la validez de la comparativa (diap. 13)

LA DIAPOSITIVA DICE

*«Filtros soportados: edad · posición · valor ·
pie · altura · contrato · nacionalidad · liga»*

EL CÓDIGO HACE

- `_normalize_league_name` solo reconoce claves de `LEAGUE_SYNONYMS`, como «premier league escocia».
- El prompt pide al LLM el nombre canónico «Premier league (Escocia)», que no está entre las claves y se descarta.
- Resultado en el log: `liga detectada = []` y el filtro no se aplica (762 candidatos).

QUÉ CAMBIAR

Reconocer que en 3 de las 9 consultas estándar (liga escocesa, suiza y turca) no se aplicó el filtro de liga.

```
src/scouting/agents/agent1.py      líneas 274, 405-408, 1211-1219

274     "premier league escocia": "Premier league (Escocia)", "liga escocesa": "Premier league (Escocia)",
    ...
405 def _normalize_league_name(name: str) -> str | None:
406     if not isinstance(name, str):
407         return None
408     return LEAGUE_SYNONYMS.get(_norm_text(name))
    ...
1211     if ligas:
1212         normalized_ligas = [l for l in ([_normalize_league_name(l) for l in ligas]) if l]
1213         ligas = list(dict.fromkeys(normalized_ligas))
1214         if self.DEBUG: print(f"[A1][Liga] detectadas={ligas}")
1215         if ligas and "liga" in df.columns:
1216             df = df.with_columns(pl.col("liga").cast(pl.Utf8, strict=False).alias("liga"))
1217             antes = df.height
1218             df = df.filter(pl.col("liga").is_in(ligas))
1219             if self.DEBUG: print(f"[A1][Liga] filtro: {antes} -> {df.height}")
```

```
logs_completos_tfm.txt      líneas 14, 17, 27, 30-31

14 [SUPERVISOR] domain=ok lang=es query_es=Delantero de la liga escocesa con buen juego aéreo...
    ...
17 [A1][LLM] criterios extraídos: {'edad_min': None, 'edad_max': None, 'valor_max_euros': None,
    'posiciones': ['centre-forward'], 'nacionalidad': None, 'pie_dominante': None, 'altura_min_cm': None,
    'altura_max_cm': None, 'contrato_max_anios': None, 'liga': 'Premier league (Escocia)'}
    ...
27 [A1][Liga] detectadas=[]
    ...
30 [A1][PaisLiga] filtro: 762 -> 762
31 Agente 1: Filtrados 762 jugadores tras aplicar filtros clave.
```

5 El juez no puede verificar los datos

Afecta a: preguntas de la diapositiva 12 («la fidelidad forma parte de la evaluación») y de la 13

LA DIAPOSITIVA DICE

«El juez recibió la respuesta completa... incluyendo la explicación, los candidatos seleccionados y los metadatos asociados.»

EL CÓDIGO HACE

- A GPT-4o solo le llega la explicación, los nombres de los candidatos y una línea de latencia.
- No recibe ninguna estadística: no puede saber si una cifra es inventada.
- La salida incluye «Provider / Model», así que la evaluación no es ciega.

QUÉ CAMBIAR

Presentar el criterio 2 como plausibilidad, no como verificación, y añadir la evaluación no ciega a las amenazas a la validez.

```
scouting_provider.py líneas 121-134
121     # Extraer campos del state resultante
122     explicacion = state.get("explicacion", "ERROR: Sin respuesta")
123     candidatos = [
124         r["nombre"] for r in state.get("resultados", []) if "nombre" in r
125     ]
126
127     # Consolidar en una sola cadena de texto
128     salida_partes = [
129         explicacion,
130         "",
131         "---",
132         f"Candidatos: {' '.join(candidatos) if candidatos else 'Ninguno'}",
133         f"Latencia: {latencia}s | Provider: {provider} | Model: {model_name}",
134     ]
```

```
promptfooconfig.yaml líneas 54-57
54     2. FUNDAMENTACIÓN EN DATOS: ¿La explicación se apoya en datos concretos
55     (percentiles, valores per90, puntuaciones) y NO inventa estadísticas
56     ni jugadores ficticios? Cada afirmación sobre rendimiento debe estar
57     respaldada por una métrica específica mencionada en el texto.
```

6 Reproducibilidad y cifra de la base

Afecta a: diapositiva 5 (15.347 registros) y diapositiva 7 (infraestructura reproducible)

LA DIAPOSITIVA DICE

«15.347 registros de jugadores (13.814 de campo · 1.533 porteros)» · «Mismo entorno para desarrollo, ejecución y evaluación experimental.»

EL CÓDIGO HACE

- scouting-etl (servicio etl de Compose) llama a main(skip_fotmob=...), pero main() no acepta argumentos.
- Último merge registrado: 15.066 jugadores. El README dice 14.476.
- La imagen no instala langchain-community, que usa el proveedor de Promptfoo.

QUÉ CAMBIAR

Confirmar la cifra de la base evaluada y unificarla en la diapositiva, la memoria y el README. Matizar el «mismo entorno».

```
src/scouting/etl/main_etl.py líneas 30, 78-86
30 def main():
    ...
78 def cli():
79     # reusa tu main actual: parsea argumentos y llama a las fases
80     import argparse
81     parser = argparse.ArgumentParser()
82     parser.add_argument("--skip-fotmob", action="store_true")
83     parser.add_argument("--skip-tm", action="store_true")
84     parser.add_argument("--skip-merge", action="store_true")
85     args = parser.parse_args()
86     main(skip_fotmob=args.skip_fotmob, skip_tm=args.skip_tm, skip_merge=args.skip_merge) # o como se llame tu función
```

```
src/scouting/etl/main_etl.log líneas 20731-20732
20731 === [3/4] MERGE ===
20732 ✓ TOTAL FINALES: 15066 jugadores (porteros: 1527, jugadores: 13539)
```

```
Dockerfile líneas 43-48
43 RUN pip install \
44     python-dotenv langgraph langchain-openai \
45     plotly kaleido \
46     sentence-transformers faiss-cpu polars unicode aiohttp bs4 requests rapidfuzz \
47     mplsoccer==1.2.2 Pillow>=10.3 matplotlib>=3.8 numpy>=1.26 \
48     brotli lxml curl_cffi langchain-groq
```

```
scouting_provider.py líneas 98
98 from langchain_community.callbacks.manager import get_openai_callback
```

Antes de la defensa

1

Corregir las diapositivas 8, 9, 10 y 11

Agente 2 sin LLM y top_k = 5.000.

2

Cambiar o explicar el ejemplo de la diap. 10

Comprobar en Promptfoo qué liga salió para la consulta 9.

3

Preparar la respuesta sobre el filtro de liga

3 de 9 consultas estándar sin filtro de liga.

4

Ajustar las amenazas a la validez

Juez sin datos de origen y evaluación no ciega.

5

Unificar la cifra de la base

Diapositiva, memoria y README con el mismo número.

No se ha modificado el código: los experimentos se hicieron con esta versión, así que lo coherente es ajustar las diapositivas.